

Ridge, Lasso and ElasticNet

Hakan Gogtas

Ridge regression

This package provides extremely efficient procedures for fitting the Ridge, Lasso, and ElasticNet regularization paths for linear regression, logistic and multinomial regression models, Poisson regression, and the Cox model. The algorithm is extremely fast and exploits sparsity in the input matrix where it exists. A variety of predictions can be made from the fitted models. The main function in the package is `glmnet()`. This function fits a generalized linear model (GLM) via penalized maximum likelihood. The regularization path is computed for the Ridge, Lasso, or ElasticNet penalty at a grid of values for the regularization parameter `lambda`. Can deal with all shapes of data, including very large sparse data matrices.

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
BodyFat<- read.csv("Body Fat.csv")
#need to define X and Y matrix for glmnet
#take out Y and law indicator from data for X (all independent variable)
#to get all the Xs (independent Variables)
x <- model.matrix(Y~., BodyFat)[-c(1)]
#our dependent variable
y <- BodyFat$Y
RidgeMod <- glmnet(x, y, alpha=0, nlambda=100,lambda.min.ratio=0.0001)

#coefficients for the smallest lambda
coef(RidgeMod, s = min(RidgeMod$lambda))
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##              s=0.4370159
## (Intercept) -10.0097587
## X1           0.4358802
## X2           0.4385841
## X3          -0.1183033
```

```
#coefficients for the largest lambda
coef(RidgeMod, s = max(RidgeMod$lambda))
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##              s=4370.159
## (Intercept) 2.019500e+01
## X1          8.658450e-37
## X2          8.651986e-37
## X3          2.014431e-37
```

In the `glmnet` function use the following arguments:

`-nlambda=100`: Set the number of `lambda` values (the default is 100)

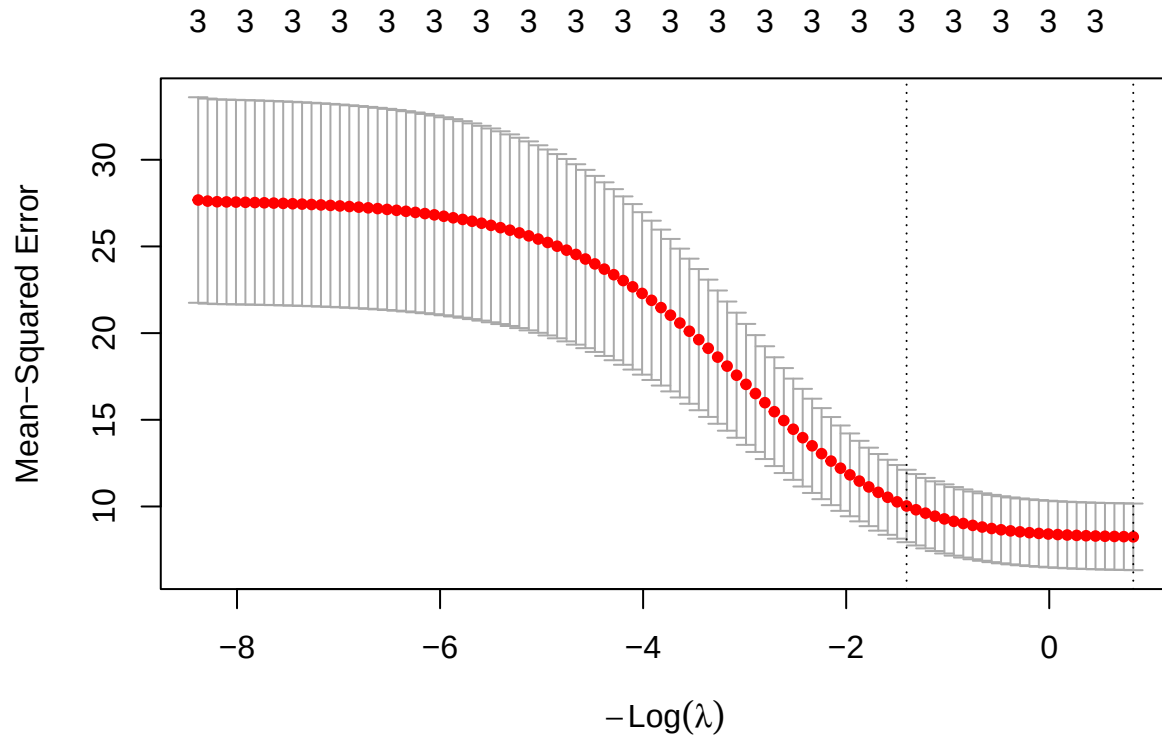
-lambda.min.ratio=0.0001: Set the smallest value for lambda, as a fraction of lambda.max, the (data derived) entry value (that is, the smallest value for which all coefficients are zero)

we will use cross validation to select the best lambda

```
CvRidgeMod <- cv.glmnet(x, y, alpha=0, nlambda=100, lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
par(mfrow=c(1,1))
plot(CvRidgeMod)
```



```
#get the optimal lambda
best.lambda.ridge <- CvRidgeMod$lambda.min
best.lambda.ridge
```

```
## [1] 0.4370159
```

```
coef(CvRidgeMod , s = "lambda.min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.min
## (Intercept) -10.0097587
## X1          0.4358802
## X2          0.4385841
## X3         -0.1183033
```

The figure includes the cross-validation curve (red dotted line) and upper and lower standard deviation curves along the λ sequence (error bars). In the beginning of the procedure (to the right of the figure), the MSE is very high, and the coefficients are restricted to be too small; and then at some point, it kind of levels off. This seems to indicate that the full model is doing a good job.

There are two vertical lines: one is at the minimum, and the other vertical line is within one standard error of

the minimum. The second line is a slightly more restricted model that does almost as well as the minimum, and sometimes, we'll go for that. These lines then lie at two lambda values:

-lambda.min is the value of λ that gives the minimum mean cross-validated error

-lambda.1se, gives the most regularized model such that the error is within one standard error of the minimum

At the top of the plot, you actually see how many nonzero variables' coefficients are in the model. There are all six variables in the model (five variables, plus the intercept), and no coefficient is zero.

Once we have the best lambda, we can use predict to obtain coefficients:

```
predict(RidgeMod, s=best.lambda.ridge, type="coefficients")[1:4, ]
```

```
## (Intercept)          X1          X2          X3
## -10.0097587   0.4358802   0.4385841  -0.1183033
```

This is the best regression model for our data.

Lasso Regression

To perform Lasso regression, we will use the same dataset. On the R function below $\alpha = 1$ for Lasso regression. Everything is very similar.

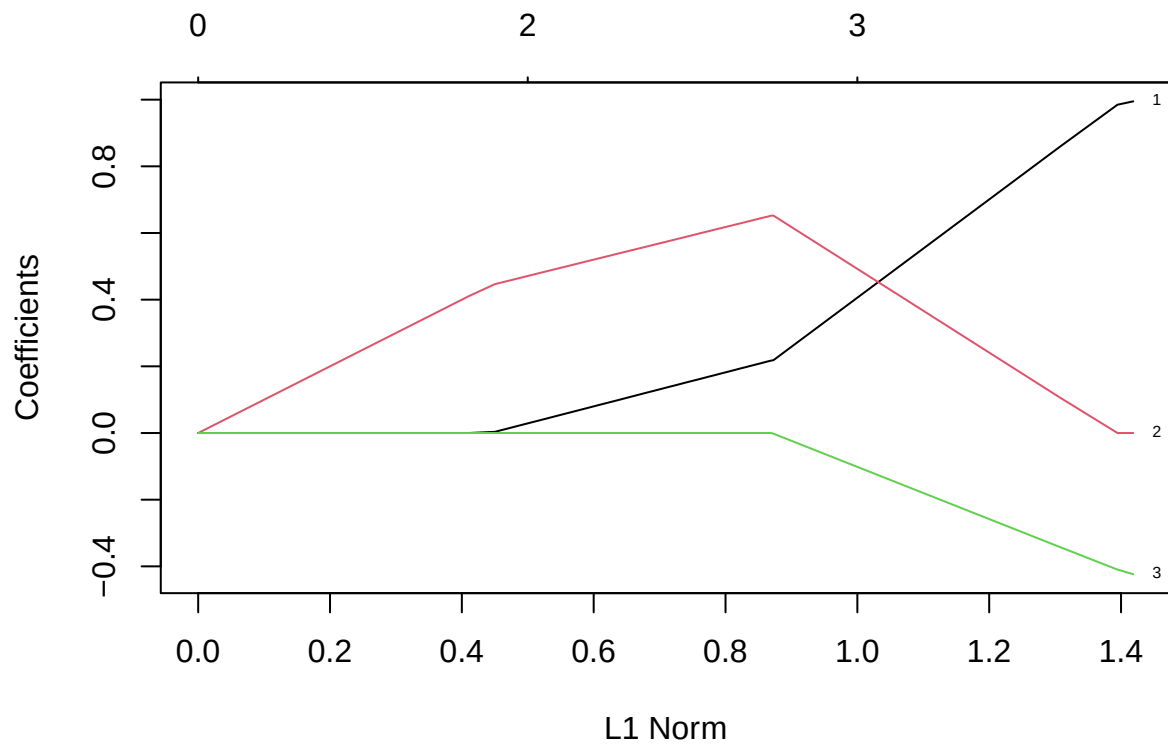
```
LassoMod <- glmnet(x, y, alpha=1, nlambda=100, lambda.min.ratio=0.0001)
#coefficients for the smallest lambda
coef(LassoMod, s = min(LassoMod$lambda))
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##           s=0.0149917
## (Intercept)  6.7197922
## X1          0.9949266
## X2          .
## X3         -0.4236571
```

```
#coefficients for the largest lambda
coef(LassoMod, s = max(LassoMod$lambda))
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##           s=4.370159
## (Intercept)  20.195
## X1          .
## X2          .
## X3          .
```

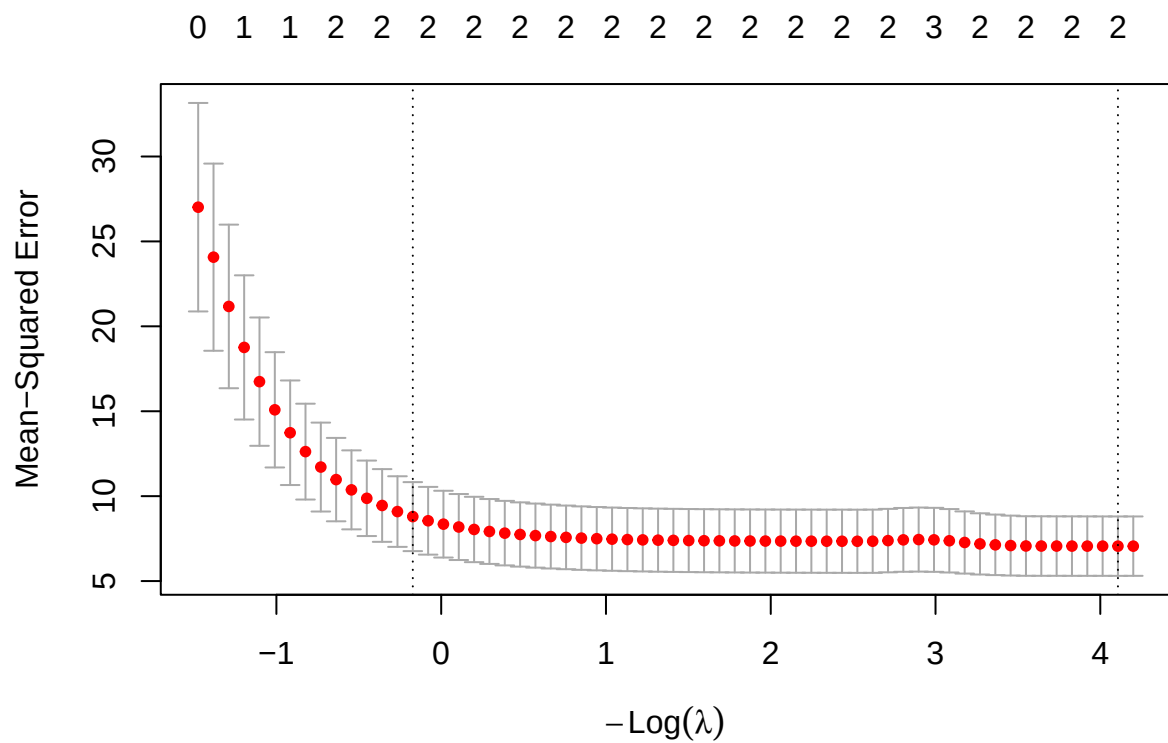
```
plot(LassoMod, xvar="norm", label=TRUE)
```



```
CvLassoMod <- cv.glmnet(x, y, alpha=1, nlambda=100, lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
plot(CvLassoMod)
```



```
#get the optimal lambda
best.lambda.lasso <- CvLassoMod$lambda.min
best.lambda.lasso
```

```
## [1] 0.01645337
```

```
coef(CvLassoMod, s = "lambda.min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.min
## (Intercept) 6.7127883
## X1          0.9943750
## X2          .
## X3         -0.4228981
```

We have confirmed that the Lasso method is able to make a selection of variables. Ultimately, we can say that both Lasso and Ridge balance the trade-off bias-variance with the choice of λ . Lasso implicitly assumes that part of the coefficients are zero, or at least not significant. Lasso tends to have a higher performance than Ridge in cases where many predictors are not actually tied to the response variables. In opposite cases, the Ridge tends to have better performance. Both approaches can be compared by cross-validation.

ElasticNet

```
EnetMod <- glmnet(x, y, alpha=0.5, nlambda=100,lambda.min.ratio=0.0001)
CvElasticnetMod <- cv.glmnet(x, y,alpha=0.5,nlambda=100,lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
best.lambda.enet <- CvElasticnetMod$lambda.min
best.lambda.enet
```

```
## [1] 0.7780763
```

```
coefficients(EnetMod,s=best.lambda.enet)
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##           s=0.7780763
## (Intercept) -12.0881923
## X1          0.2832207
## X2          0.4908402
## X3          .
```

```
coef(CvElasticnetMod, s = "lambda.min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.min
## (Intercept) -12.0881923
## X1          0.2832207
## X2          0.4908402
## X3          .
```

Let us evaluate the model performances of Linear Regression, Ridge, Lasso and Elastic Net models on the train data sets. The same code can be used for the train data set.

```
f<-lm(Y~.,data=BodyFat)
y_hat.ridge <- predict(RidgeMod, s = best.lambda.ridge, newx = x)
y_hat.lasso <- predict(LassoMod, s = best.lambda.lasso, newx = x)
y_hat.enet <- predict(CvElasticnetMod, s = best.lambda.enet, newx = x)
```

```

###(## Model Performance on the Train Data
library(caret)

## Loading required package: ggplot2
## Loading required package: lattice

## Linear Regression Model
Predicted<-f$fitted.values
ModelTrain.reg<-data.frame(obs = BodyFat$Y, pred=Predicted)
reg<-defaultSummary(ModelTrain.reg)

## Ridge Regression Model
Predicted<-c(y_hat.ridge)
ModelTrain.ridge<-data.frame(obs = BodyFat$Y, pred=Predicted)
ridge<-defaultSummary(ModelTrain.ridge)

## Lasso Regression Model
Predicted<-c(y_hat.lasso)
ModelTrain.lasso<-data.frame(obs = BodyFat$Y, pred=Predicted)
lasso<-defaultSummary(ModelTrain.lasso)

## Elastic Net Regression Model
Predicted<-c(y_hat.enet)
ModelTrain.enet<-data.frame(obs = BodyFat$Y, pred=Predicted)
enet<-defaultSummary(ModelTrain.enet)

data.frame(rbind(reg,ridge,lasso,enet))

##           RMSE  Rsquared    MAE
## reg    2.218162 0.8013586 1.885628
## ridge  2.340010 0.7806201 1.936593
## lasso  2.301675 0.7861426 1.924385
## enet   2.419202 0.7765297 2.024189

```

Based on model performance metrics above, there are no clear differences among ridge, lasso and elastic net models. From the simplicity perspective and assuming that there are no constraint for us to drop variables, i would choose lasso regression model.